

Everything standing between this build and a first real order

The commerce build is substantially complete, passes its suites, and has never been deployed. This edition re-ran what could be re-run and corrects the single most important claim in the 31 August report, which was wrong in a way that matters.

Stack Medusa 2.18.0 • Next.js 16.3.2 • React 19.2.8 • Postgres 17
Surface 20 storefront routes • 36 API routes • 8 admin screens Tests 537 re-run and green today • 0 unrunnable
Deployed nowhere

CORRECTION 1 The fail-closed boot guard is never called. It is dead code	HARD BLOCKERS 8 4 keys, R2, hosting, managed Postgres, secrets	CANNOT BE CODED 4 IOSS, GDPR Art. 27, GPSR person, licensing. Start today	ONE SUPPLIER EMAIL 5 gaps Fibre, origin, HS codes, measurements, print format
---	---	--	--

READ THIS FIRST — AND IT REVERSES THE LAST REPORT

The 31 August report opened by saying production could not boot because Cloudflare R2 is marked critical. That is wrong, and the truth is worse. `assertConfigured()` — the function that is supposed to refuse a production start when a critical key is missing — **is never called anywhere**. Grepping the repository outside `node_modules` returns exactly two hits: its own definition at `src/integrations.ts:113`, and a mention of itself in its own doc comment.

So the fail-closed rule that file spends fifteen lines documenting is not in force. `criticalInProduction: true` currently changes one thing: a number on the `/admin/integrations` dashboard. A production container will start happily with **no Stripe Tax, no Shippo and no R2**, and begin taking orders with tax uncalculated and no ability to buy a label.

Two guards *are* real, and they are the only two: a missing `STRIPE_WEBHOOK_SECRET` throws from `medusa-config.ts:14`, and a missing `RESEND_API_KEY` throws from the Resend module constructor. Wiring `assertConfigured()` into boot is a one-line change and should happen before anything else on this list.

00

What was actually re-run today

Every suite was executed on 4 September 2026, including the integration suite against a real Postgres 17 and Redis 7 — the Docker daemon was started for the purpose. Nothing below is quoted from a previous run.

SUITE	COMMAND	RESULT	STATUS
Backend unit	<code>npm run test:unit</code>	215 passed , 10 suites, 2.5s	VERIFIED
Backend typecheck	<code>npx tsc --noEmit</code>	Clean, no errors	VERIFIED
Storefront unit	<code>npm test</code> (vitest)	63 passed , 5 files	VERIFIED
Storefront typecheck	<code>npm run typecheck</code>	Clean, no errors	VERIFIED

SUITE	COMMAND	RESULT	STATUS
Tools — parsing	<code>python3 -m unittest discover</code>	82 passed	VERIFIED
Contrast — WCAG AA	<code>contrast_check.py</code>	All colour pairs pass	VERIFIED
Accessibility self-test	<code>a11y_selftest.py</code>	24 of 24 correct — the checks can still fail	VERIFIED
Backend integration	<code>npm run test:integration:http</code>	177 passed , 12 suites, 236s against a real Postgres 17 + Redis 7	VERIFIED

ON THE HEADLINE NUMBER

The README's claim of **537 tests is exact**, and every one of them passed today: 215 unit, 177 integration, 63 storefront, 82 tools. An earlier static count in this document put the integration figure at 174 by grepping for `it(` ; three cases are generated rather than written out literally, so the grep undercounted and the README was right. The suite was run rather than counted.

01

Code — front end

Next.js 16.3.2 on React 19.2.8. **Twenty page routes**, all rendering, typecheck clean, 63 unit tests green. Stripe is wired through `@stripe/react-stripe-js`, which is what puts the card fields on our own domain.

The routes that exist

`/` · `/jerseys` · `/jerseys/[handle]` · `/collections/[handle]` · `/cart` · `/checkout` · `/order/[id]` · `/track` · `/request` · `/returns` · `/shipping` · `/size-guide` · `/contact` · `/policies/[slug]` · `/privacy-choices` · `/account` and its five sub-routes (login, register, forgot, reset, orders).

ITEM	STATE	WHEN
Screen-reader pass	The mechanical audit covers 25 pages and checks landmarks, radio grouping, live regions and the skip link. Nobody has listened to VoiceOver or NVDA work the buy box or checkout, and nobody has done a keyboard-only traversal. This is the one front-end item that cannot be automated away.	BEFORE LAUNCH
Size measurements	The <code>MEASUREMENTS</code> table is empty, so <code>/size-guide</code> says so rather than printing a generic chart. Under a final-sale policy a wrong size is the customer's loss, which makes this the highest-value line in the supplier email.	BEFORE LAUNCH
Favicon and brand assets	The default Next.js favicon is still in place; <code>public/</code> was emptied of the starter SVGs and nothing replaced them.	BEFORE LAUNCH
Product imagery	67% of products have exactly one photograph. A deliberate decision (\$13.6) with a stated cost: fit and appearance surprises drive apparel returns. Photograph the top sellers once there is order data to pick them.	AFTER LAUNCH
Personalisation teaser	The homepage block is a placeholder. The preview itself is live on every eligible product page; a teaser that cannot lead to a finished order is not worth the space until the print-file format exists.	AFTER LAUNCH
Search quality	Free-text search is <code>ILIKE '%...%'</code> , which cannot use an index. Fine at 4,300 products, a real problem at 40,000 or under load.	AFTER LAUNCH

Code — back end

Medusa 2.18.0. **36 API route files** — 17 store, 17 admin, plus health and media — and **8 admin screens** built as dashboard extensions. Four custom modules (`catalog` , `resend` , `tax-stripe` , `fulfillment-shippo`), eleven migrations, 215 unit tests, typecheck clean.

Correctness and scale

ITEM	DETAIL	WHEN
Boot guard not wired	<code>assertConfigured()</code> is exported and never called. See page one. A one-line fix that restores the fail-closed behaviour the codebase already documents and tests around.	BLOCKER
Event bus and locking	No Redis event-bus, locking, cache or workflow-engine module is registered in <code>medusa-config.ts</code> — setting <code>projectConfig.redisUrl</code> does not switch them. Medusa announces it at every boot: <i>"Local Event Bus installed. This is not recommended for production."</i> Jobs run in-process, so a restart loses queued work, including order confirmation emails . Redis is already running in compose; this is registration, not new code.	BLOCKER
Print-file generator	The only genuinely blocked piece of engineering. Needs the supplier's required format — vector or raster, spot colours, template. The on-screen preview is independent and finished.	BLOCKER
In-process caches	Four endpoints cache in module memory: facets, sitemap, store reviews, collections. Correct on one instance, wrong on two — and an import stays invisible until a TTL expires.	BEFORE LAUNCH
Refund and label execution	The returns queue records decisions and deliberately does not move money or buy labels: both are irreversible external calls and Shippo's transaction endpoint is not idempotent. Somebody does that step by hand in Orders, or it needs building with an idempotency key.	BEFORE LAUNCH
Instrumentation	<code>instrumentation.ts</code> is the untouched Medusa stub — every line commented out. No OpenTelemetry, no traces. The custom error reporter is separate and does work.	BEFORE LAUNCH
Public write endpoints	Rate limiting exists as a tested module. <code>/store/jersey-requests</code> is a public POST that writes rows and needs it applied before it faces the internet.	BEFORE LAUNCH
Inventory	<code>manage_inventory</code> is false on every variant. A decision for a sourcing model, not an oversight — but it means nothing can ever be out of stock, including things the supplier cannot get.	AFTER LAUNCH
Invoice PDFs	Required for EU B2C sales only. A US-first launch does not need them.	AFTER LAUNCH

WORTH NOT RE-LITIGATING

Error handling is registered as `config.errorHandler` and delegates to Medusa's own handler explicitly, because `next(err)` fell through to Express and answered a routine 400 with a 500 HTML page containing a stack trace. Request ids are attached inbound and returned on every response. The Sentry reporter scrubs secrets by key name *and* by value shape, because either alone misses cases.

APIs, keys and integrations

Nine integrations are declared in `src/integrations.ts`. Five are flagged `criticalInProduction`. Read the *Boot* column carefully: it is what actually happens today, not what the flag claims.

INTEGRATION	IMPLEMENTATION	BOOT BEHAVIOUR TODAY	WHEN
Stripe PAYMENTS	Medusa's <code>payment-stripe</code> provider. Path proven 7 of 8 steps through the Store API with no browser involved; the last step needs a test key.	Throws without <code>STRIPE_WEBHOOK_SECRET</code> . Measured: with it empty the webhook returns HTTP 200 to unsigned <i>and</i> forged payloads.	BLOCKER
Resend TRANSACTIONAL EMAIL	Own module. 9 templates wired to 6 triggers. Renders and logs without a key.	Throws in production without <code>RESEND_API_KEY</code> . Order confirmations do not exist until it is set.	BLOCKER
Stripe Tax US NEXUS, EU VAT	Own module, tested — including that it sends cents rather than dollars, and that it distinguishes "no tax owed" from "tax not switched on". One live HTTP call untested.	Starts anyway. Flagged critical, guard never runs. Off unless <code>STRIPE_TAX_ENABLED=true</code> ; billing is 0.5% of volume.	BLOCKER
Shippo LABELS, TRACKING, CUSTOMS	Own module. Rating is already live off our own rate card and needs no account. Only labels, tracking and customs documents need the key.	Starts anyway. Flagged critical, guard never runs.	BLOCKER
Cloudflare R2 IMAGE STORAGE	No implementation at all. <code>MEDIA_BACKEND=r2</code> appears in one comment, one admin string, the compose file and the deploy template — and is read by no code. The <code>R2_*</code> variables appear only in the registry that lists them. Images are served from Postgres and work.	Starts anyway. Decide: build the adapter, or set the flag to <code>false</code> and stop calling it critical.	BLOCKER
Sentry ERRORS	Custom reporter over <code>fetch</code> against Sentry's envelope API — no SDK, deliberately, because the SDK patches <code>http</code> and <code>async_hooks</code> at import and Medusa's worker split is where that goes wrong. Fingerprinting and scrubbing work without a DSN.	Optional. Nothing is being watched until <code>SENTRY_DSN</code> is set.	BEFORE LAUNCH
PostHog ANALYTICS	Consent-gated loader in the storefront: nothing loads before a positive choice and nothing loads at all under Global Privacy Control. Setting the key also adds the host to the CSP allow-list.	Optional. No funnel data until the key is set.	BEFORE LAUNCH
Typesense SEARCH	Registered with no implementation. Search falls back to SQL <code>ILIKE</code> . A key today would do nothing.	Optional.	AFTER LAUNCH
Omnisend MARKETING EMAIL	Registered with no implementation. Newsletter subscribers are being collected so the list exists whenever a provider is chosen.	Optional.	AFTER LAUNCH

The payment architecture decision, restated because it sets the compliance budget

Medusa 2.18's Stripe provider is **PaymentIntents plus Elements**, not Checkout Sessions. The research memo recommended Checkout Sessions; that path is not on offer. The consequence is not cosmetic: the card fields are an *iframe* on *our* domain, so the checkout page is in PCI scope and the assessment moves from **SAQ A to SAQ A-EP** — requirements 6.4.3 (script inventory and integrity) and 11.6.1 (tamper detection) become materially heavier. The CSP has already been written for SAQ A-EP with every entry justified and is now derived from the environment rather than pinned to localhost.

The functional argument for Checkout Sessions mostly evaporates anyway, because Medusa already owns cart, tax, discounts, shipping and addresses — which was the reason to prefer Checkout in the first place. What is given up is Stripe-hosted conversion tuning, Link, and Adaptive Pricing.

04

Infrastructure — none of this exists

This is the largest remaining block of work and the least interesting. `docker-compose.yml` is production-*shaped* — built images, real Postgres and Redis, health checks, no bind mounts, `NODE_ENV: production` — and has never been run against a real environment. `.env.deploy` does not exist. Both services have a `Dockerfile` and CI builds both images; nothing consumes them.

NEED	TODAY	WHEN
Hosting	Nothing chosen. Two containers plus Postgres and Redis.	BLOCKER
Managed Postgres	Docker on one laptop, and the daemon was down today. Needs automated backups, point-in-time recovery and a <i>tested</i> restore drill. Restore time is the single number that should trigger moving images out of the database — 728 MB of image bytes ride along with every backup.	BLOCKER
Secret manager	<code>.env</code> files on a laptop. Needs a real store and a rotation policy before any live key is issued.	BLOCKER
TLS and domain	Everything is localhost. The CSP now derives from <code>NEXT_PUBLIC_MEDUSA_URL</code> , so this is configuration rather than code — but it is untested against a real origin.	BLOCKER
Redis wired for real	Running in compose and not registered as Medusa’s event bus, cache or lock provider. See §02.	BLOCKER
CI to staging	CI runs six jobs on every push — tools, backend unit, backend integration against a real Postgres 17 and Redis 7, storefront, and both image builds. It deploys nothing, and there is no staging environment to deploy to.	BEFORE LAUNCH
Rollback	Migrations are forward-only in practice; no rollback has ever been rehearsed.	BEFORE LAUNCH
Monitoring and on-call	Nothing is watching. No uptime checks, no alerting, no log aggregation. You are the rotation.	BEFORE LAUNCH
CDN	Images are content-addressed and served <code>immutable</code> with a one-year max-age, so they are CDN-ready — and no CDN is in front of them. Every image request currently holds a database connection and competes with checkout.	BEFORE LAUNCH
Node pinning	Node 22.23.2 via a <code>PATH</code> export. Medusa supports 20/22 LTS only and this machine defaults to unsupported Node 25. Pinned correctly in CI and in the images; unpinned locally.	BEFORE LAUNCH

05

Legal and compliance

THE FOUR THAT NO CODE CAN CLOSE

Three gate EU revenue outright and one gates personalisation, which is the entire commercial argument for owning this stack. They have lead times measured in weeks, they cost nothing to begin, and they can all start today in parallel with everything else. The storefront already refuses EU addresses and names which appointment is missing, so the *build* is not waiting on them — the revenue is.

REQUIREMENT	CONSEQUENCE WHILE MISSING	WHEN
IOSS registration + EU intermediary	Import VAT on EU B2C consignments cannot be collected correctly. EU sales blocked.	BLOCKER
GDPR Article 27 EU representative	Required to sell into the EU from the US. EU sales blocked.	BLOCKER
GPSR EU Responsible Person	Without one, apparel cannot lawfully be placed on the EU market at all.	BLOCKER
Licensing position	Unresolved. Printing a player's name to order raises the question more sharply than selling a stock shirt does. This gates personalisation <i>going live</i> , not being built.	BLOCKER
US economic nexus registrations	Stripe Tax calculates; it does not register. Nexus reaches 45 states.	BEFORE LAUNCH
Legal review of the policies	Privacy, terms, refunds and shipping are published with a visible draft banner. A refund policy is a contract term; the banner is honest, not sufficient.	BEFORE LAUNCH

Trader identity — three disclosures still outstanding

Eight identity fields are enumerated in one place (`storefront/lib/site.ts`) with the legal basis for each. The policy pages call `pendingEntityFields()` and render every unset field as a **labelled gap with the reason it is required**, rather than omitting a required disclosure or inventing a value. Five are filled from the live store's own published contact information. Three are empty:

- **Registered postal address** — required on the storefront and on every invoice. The live store publishes none either.
- **Privacy / data-subject request channel** — GDPR Art. 15–22 and the US state privacy laws all require one.
- **A domain mailbox**. Support currently resolves to a personal Gmail address, and the trading entity (*Crux Christi*) differs from the brand (*Find Any Jersey*). Both are lawful; neither is a good long-term answer.

ALREADY DONE, AND WORTH NOT RE-LITIGATING

Consent is opt-in with Global Privacy Control honoured silently and without a banner. The US state opt-out page exists with a working control. The CSP is written for PCI SAQ A-EP with every entry justified. Reviews carry provenance and cannot be presented as verified when they are not. No compare-at price, countdown or stock claim appears anywhere the data cannot support it. WCAG AA contrast is enforced by a script that runs in CI and has already caught one real 1.27:1 failure.

06

Data and supplier inputs

ONE EMAIL CLOSES FIVE GAPS

Fibre composition, country of origin, HS codes, care instructions and chest/length measurements. Four are regulatory and one is the highest-leverage line on the storefront under a final-sale policy. The `jersey_detail` table has eight regulatory columns and **every one is null on every product**.

FIELD	COVERAGE	BLOCKS	WHEN
Fibre composition	none	EU Textile Regulation	BLOCKER

FIELD	COVERAGE	BLOCKS	WHEN
Country of origin	none	Customs declarations	BLOCKER
HS code	none	Cross-border shipping	BLOCKER
EU responsible person	none	GPSR — EU market entry	BLOCKER
Print-file format	unknown	Every queued personalisation	BLOCKER
Size measurements	none	Size guide, and returns we cannot accept	BEFORE LAUNCH
Supplier cost per personalisation	unknown	Confirming the 55% margin assumption — not the build	BEFORE LAUNCH
Patch list per league	none	The patch tier is built and offers nothing until PATCHES_<LEAGUE> is set	AFTER LAUNCH

Catalog

The database is not running today, so these are carried forward from the 29 August sync and the 31 August audit rather than re-counted. What *was* re-checked is the source data on disk, which agrees.

MEASURE	VALUE	SOURCE
Live-store snapshot on disk	4,318 products	re-counted today VERIFIED
Imported catalog	4,323 products / 29,002 variants	CARRIED FWD
Catalogue price	\$65.99 flat, every product, every region	set and verified today VERIFIED
Custom blanks	111, printing included at \$89.99 — 69 from the NFL snapshot plus 42 living inside the main catalog with ordinary-looking handles	re-counted today VERIFIED
Curated collections	11	re-counted today VERIFIED
Flagged needs_review	157	CARRIED FWD
No team resolved	113 — excluded from the team facet until reviewed	CARRIED FWD
Exactly one image	67%	CARRIED FWD
Imported reviews	84, average 4.94 — matching the source aggregate exactly	CARRIED FWD
... attached to a product	17	CARRIED FWD
In our catalog, not on the live store	1,076 — left alone deliberately	CARRIED FWD

PRICING, SET 4 SEPTEMBER

The catalog now carries **one price: \$65.99 USD** — 4,212 products and 28,221 variants, identical in all five regions. It replaced a spread of \$30–\$119.99. All five regions were already `usd` with no price lists and no currency conversion, so the same number is what a shopper sees in Sydney and in Ohio; only shipping and tax vary by destination.

Two things were deliberately left alone: the **111 custom blanks at \$89.99**, which ship with the name and number included, and the **personalisation add-ons** (\$9.99 number, \$14.99 name, \$19.99 bundle, \$7.99 patch). Pricing a blank the same as a plain shirt would have made the bundle unsellable.

The `needs_review` flags are mostly unresolvable ambiguity rather than error: "Los Angeles Basketball" is the Lakers or the Clippers, and a wrong team is worse than a missing one. The 67 reviews that did not attach name no product, or name a shirt we no longer stock — there is no more per-product data in existence, so this is not a migration failure to go back and fix.

07

The order to do it in

Grouped by what unblocks what, not by size. Nothing in the first group takes long, and nothing in the later groups can start without it.

This week — costs nothing, unblocks everything

1. **Wire `assertConfigured()` into boot** and re-check every `criticalInProduction` flag against what is actually implemented. R2 is flagged critical and does not exist; either build the adapter or set the flag to `false`. One line, and it decides whether the next fifteen items are guarded or merely documented.
2. **Start the four legal appointments.** IOSS and an EU intermediary, the Article 27 representative, the GPSR responsible person, and a licensing opinion. Weeks of lead time, nothing to build, nothing to wait for.
3. **Send the supplier email.** Five gaps, one message: fibre composition, country of origin, HS codes, care instructions, chest and length measurements — plus the print-file format and the per-personalisation cost.
4. **Register Redis** as Medusa's event bus, cache and lock provider. It is already running; this is configuration.

Next — the deployment block

5. Choose hosting and stand up managed Postgres with automated backups. **Run a restore drill and time it** — that number decides whether images stay in the database.
6. Stand up a secret manager. No live key touches a laptop.
7. Domain, TLS, and the first real origin. Verify the derived CSP against it.
8. Issue the four blocking keys: Stripe live plus webhook secret, Resend, Stripe Tax, Shippo. Finish the eighth step of the payment path first, in test mode.
9. Extend CI to deploy to staging. Rehearse one rollback before you need one.
10. Uptime checks, alerting, log aggregation, and a Sentry DSN.

Before the first real order

- L1. Screen-reader pass and keyboard traversal, by a human.
- L2. Legal review of the four policy pages; fill the three outstanding trader fields.
- L3. Rate-limit the public write endpoints.
- L4. Move the four in-process caches to Redis, or accept a single instance and say so.
- L5. Decide the refund and label execution path: by hand in Orders, or built with an idempotency key.
- L6. US nexus registrations, starting with the home state.

Three things to decide, not just do

1. The fail-closed design was never switched on

This is worth more than the one-line fix. The codebase argues carefully, in comments and in tests, that a silently disabled integration is worse than a container that refuses to start — and then never calls the function that enforces it. Two module constructors happen to guard themselves, which is why the gap was invisible: boot *does* fail without Stripe or Resend, so it looks like the system works. It does not. Decide whether the rule is real, and if it is, make the flag mean something.

2. R2 is a five-minute decision that has been open for a week

Images in Postgres genuinely work, and the reasoning behind them is sound: WebP at q78 capped at 1400px is about 410 MB rather than 3.09 GB, one canonical asset with derivatives rendered on request, content-addressed and served immutable. The stated trigger for moving them out was restore time. Nobody has run a restore, so nobody has that number. Either accept Postgres and drop the R2 flag, or build the adapter — but stop carrying a critical dependency that no code reads.

3. The business case still rests on personalisation, and personalisation is gated on licensing

Cost never justified leaving Shopify; capability did, and the capability is name-and-number personalisation. That module is built, tested with 45 unit cases, and priced through the pricing engine rather than the browser. It is blocked on two things neither of which is engineering: the supplier's print-file format, and a licensing position on printing a player's name to order. Two personalisations are already queued in the database and nothing can print them.

WHERE THIS STANDS OVERALL

The interesting work is done. What remains is one unwired guard, a deployment that has never happened, four appointments with lead times, one supplier email, and a human listening to a screen reader. None of it is hard. All of it is sequential, and the first four items on the list cost nothing and can start today.